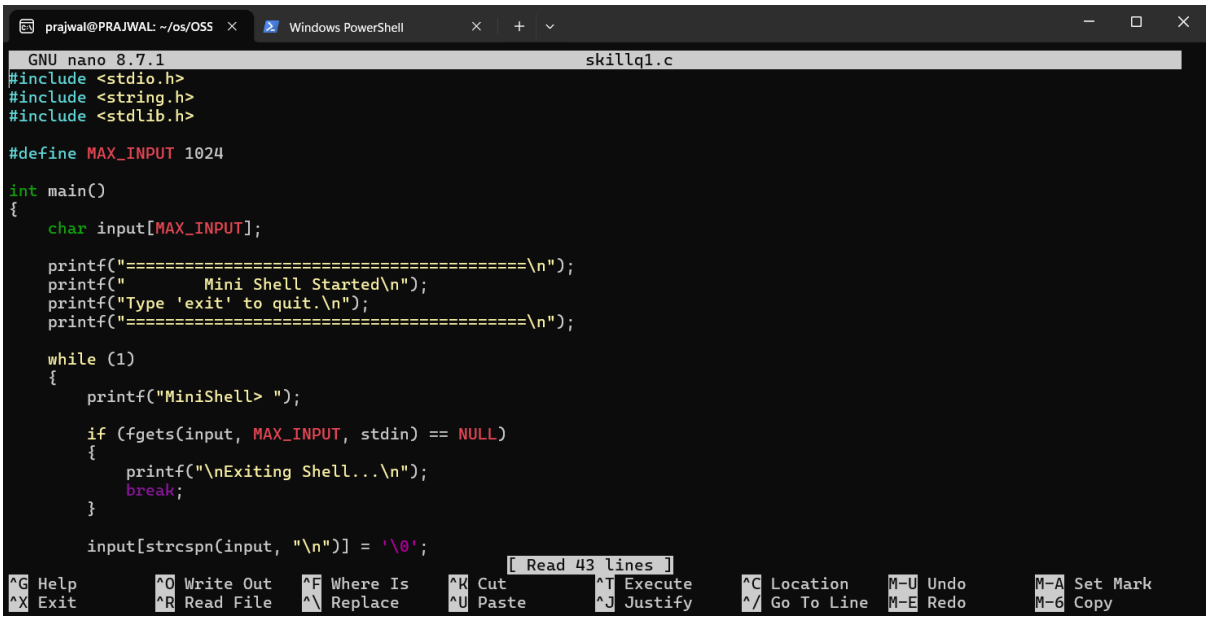
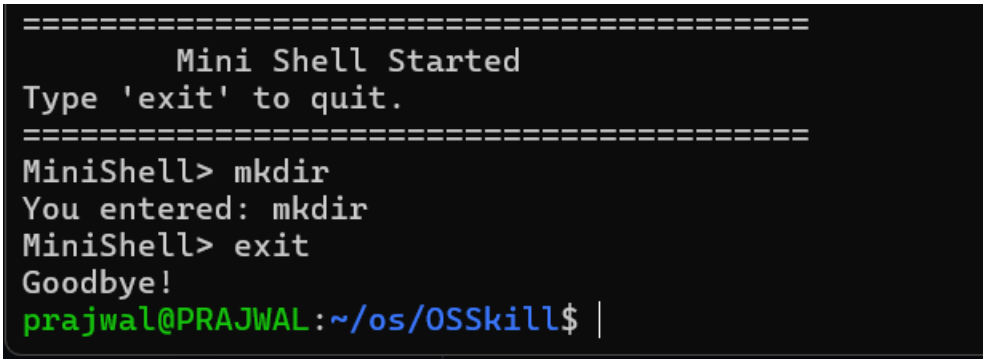
1. To Install Linux VM, Configure GCC, Setup Git Repository, Create Project Structure, Understand Shell Architecture, Build Initial Makefile

1. To Analyze process abstraction, execute fork(), understand exec() family, analyze parent-child relationships, inspect process tree, practice system call tracing





To Create Main Loop, Display Prompt, Read User Input, Handle Exit Conditions, Design Control Flow Diagram, Test Interactive Loop

2. To Capture Keyboard Input, Handle Backspace, Process Enter Key, Manage Input Buffer, Support Multi-Character Commands, Test User Interaction

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_HISTORY 100
#define MAX_INPUT 1024

int main()
{
    char **history;
    char *input;
    int count = 0;

    history = (char **)malloc(MAX_HISTORY * sizeof(char *));
    if (history == NULL)
    {
        printf("Memory allocation failed!\n");
        return 1;
    }

    input = (char *)malloc(MAX_INPUT * sizeof(char));
    if (input == NULL)
    {
        printf("Memory allocation failed!\n");
        free(history);
        return 1;
```

```
prajwal@PRAJWAL:~/os/OSSkill$ ./skillq2
========== Mini Shell ==========
Type 'history' to view history.
Type '!!' to execute last command.
Type '!n' to execute command number n.
Type 'exit' to quit.

MiniShell> ls
Executed: ls
MiniShell> !!
Previous Command: ls
MiniShell> !n 1
Invalid history number.
MiniShell> ls
Executed: ls
MiniShell> history

Command History:
1 : ls
2 : ls

MiniShell> !n 1
Invalid history number.
MiniShell> !1
Command 1: ls
MiniShell> !!
Previous Command: ls
MiniShell> exit
Memory released successfully.
```

To Apply Escape Sequences, Store Command History, Navigate Previous Commands, Navigate Next Commands, Update Input Buffer, Test Recall Functionality.

3. To Allocate Buffers Dynamically, Resize Arrays, Prevent Buffer Overflow, Manage Linked Lists, Release Memory Correctly, Verify with Valgrind

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_INPUT 1024
#define MAX_TOKENS 100

int main()
{
    char input[MAX_INPUT];
    char *tokens[MAX_TOKENS];
    int tokenCount;

    printf("========== Mini Shell Parser ==========\n");
    printf("Type 'exit' to quit.\n\n");

    while (1)
    {
        printf("MiniShell> ");

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strlen(input) == 0)
```

```
========== Mini Shell Parser ==========
Type 'exit' to quit.

MiniShell> ls

Parsed Tokens:
Token 1 : ls

Command Structure:
Command  : ls
No Arguments
--------------------------------
MiniShell> pwd

Parsed Tokens:
Token 1 : pwd

Command Structure:
Command  : pwd
No Arguments
--------------------------------
MiniShell> ls -l

Parsed Tokens:
Token 1 : ls
Token 2 : -l

Command Structure:
Command  : ls
```